Лабораторная работа №4.
Работа с таймерами микроконтроллера STM32. TIM

Цель работы: научиться работать с таймерами микроконтроллера STM32.

Общие сведения:

Таймер — это устройство, которое может генерировать и обрабатывать прямоугольные импульсы не нагружая центральный процессор. У STM32 имеется несколько таймеров с разным набором функций. Разные каналы разных таймеров могут быть подключены только к определенным контактам, что следует учитывать при проектировании принципиальной схемы устройства.

Микроконтроллер STM32 в среднем имеет:

- 3-4 таймера общего назначения (TIM2, TIM3, TIM4, TIM5).
- 1 более продвинутый таймер с расширенными возможностями (TIM1).
- 2 таймера WatchDog.
- 1 SysTick Timer.
- 2-3 базовых таймера (TIM6, TIM7, TIM8)

Таймеры могут быть как 16-битные (то есть могут считать до 65535), так и 32-битные. Умеют работать с инкрементальными энкодерами и датчиками Холла, несколько таймеров можно синхронизировать между собой. Есть прерывания на разные события, а именно:

- переполнение.
- захват сигнала.
- сравнение.
- событие-триггер.

При наступлении любого из этих событий таймеры могут генерировать запрос к DMA (DMA – прямой доступ к памяти, уже скоро мы будем разбираться и с ним). Теперь немного подробнее о каждом из режимов работы таймеров.

Основные функции таймеров в STM32:

Счет времени: Таймеры могут считать временные интервалы, увеличивая или уменьшая значение счетного регистра.

Генерация ШИМ (PWM): Таймеры используются для генерации сигналов с широтно-импульсной модуляцией, которые применяются, например, для управления двигателями или яркостью светодиодов.

Захват и сравнение: Таймеры могут фиксировать момент времени, когда на входе появляется сигнал, или сравнивать текущее значение счетчика с заданным значением.

Генерация прерываний: Таймеры могут генерировать прерывания при достижении определенных условий, например, переполнения счетчика.

Синхронизация: Таймеры могут быть синхронизированы между собой или с внешними событиями.

Структура таймера STM32

Таймеры в STM32 состоят из нескольких ключевых компонентов:

- *Счетчик (Counter)*: Это основной регистр, который увеличивается или уменьшается с каждым тактовым импульсом. Разрядность счетчика зависит от модели таймера (обычно 16 или 32 бита).
- *Предделитель (Prescaler)*: Позволяет уменьшить частоту тактового сигнала, подаваемого на счетчик. Это полезно для работы с низкими частотами.
- *Регистр автоперезагрузки (Auto-reload register, ARR)*: Определяет верхний предел счета. Когда счетчик достигает значения ARR, он сбрасывается или начинает отсчет заново.
- *Каналы захвата/сравнения (Capture/Compare Channels)*: Используются для захвата внешних сигналов или сравнения текущего значения счетчика с заданным значением.
- *Регистр сравнения (Compare Register)*: Хранит значение, с которым сравнивается счетчик для генерации событий или прерываний.

Режимы работы

Таймеры могут работать в различных режимах, таких как:

- Режим счета вверх (Upcounting).
- Режим счета вниз (Downcounting).
- Режим счета вверх-вниз (Up/Down counting).
- Режим захвата (Input Capture).
- Режим сравнения (Output Compare).
- Режим ШИМ (PWM).

Принципы работы таймера

- *Тактирование*: Таймер получает тактовый сигнал от системного тактового генератора или внешнего источника. Частота тактового сигнала может быть уменьшена с помощью предделителя.
- *Счет*: Счетчик начинает увеличивать или уменьшать свое значение с каждым тактовым импульсом.
- *Сравнение*: Когда значение счетчика достигает значения, записанного в регистре сравнения, генерируется событие или прерывание.
- *Перезагрузка*: Если счетчик достигает значения ARR, он сбрасывается или начинает отсчет заново, в зависимости от настроек.

- *Генерация сигналов*: В режиме ШИМ таймер изменяет состояние выхода (например, переключает его между высоким и низким уровнем) в зависимости от значения счетчика и регистра сравнения.

Пример использования таймера для генерации задержки

1) Настроить предделитель, чтобы уменьшить частоту тактового сигнала.
2) Установить значение ARR, чтобы определить период счета.
3) Запустить таймер.
4) Дождаться, пока счетчик достигнет значения ARR, что будет означать истечение заданного временного интервала.

Регистр TIM_CR1

Регистр управления таймером

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Бит 0. CEN (Counter ENable):
Этот бит включает или выключает таймер.

- Если бит установлен в 1, таймер начинает считать.
- Если бит сброшен в 0, таймер останавливается.

Бит 1. UDIS (Update DIStable):
Этот бит управляет генерацией событий обновления (Update Events, UE).

- Если UDIS = 0, события обновления генерируются при переполнении счетчика.
- Если UDIS = 1, события обновления запрещены.

Бит 2. URS (Update Request Source):
Этот бит определяет, какие события могут вызывать обновление.

- Если URS = 0, обновление может быть вызвано любым событием (например, переполнением, триггером).
- Если URS = 1, обновление вызывается только при переполнении счетчика.

Бит 3. OPM (One Pulse Mode):
Этот бит включает одиночный режим работы таймера.

- Если OPM = 0, таймер работает непрерывно.
- Если OPM = 1, таймер останавливается после следующего обновления.

Бит 4. DIR (DIRection):

Этот бит задает направление счета:

- DIR = 0: Счет вверх (Upcounting).
- DIR = 1: Счет вниз (Downcounting).

Биты 6-5. CMS (Center-aligned Mode Selection):
Эти биты определяют режим выравнивания счетчика:

- 00: Счетчик считает вверх или вниз (в зависимости от DIR).
- 01: Центрированный режим 1 (счетчик считает вверх и вниз, переключаясь на границе).
- 10: Центрированный режим 2 (счетчик считает вниз и вверх, переключаясь на границе).
- 11: Центрированный режим 3 (счетчик считает вверх и вниз, переключаясь на середине).

Бит 7. ARPE (Auto-reload preload enable):
Этот бит управляет буферизацией регистра ARR:

- ARPE = 0: Регистр ARR обновляется сразу.
- ARPE = 1: Регистр ARR обновляется только после следующего обновления.

Бит 8. CKD (Clock Division):
Эти биты задают делитель тактовой частоты для таймера:

- 00: Нет деления.
- 01: Тактовая частота делится на 2.
- 10: Тактовая частота делится на 4.

TIM_CR2
TIM_CR2 – регистр управления 2

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								TI1S	MMS[2:0]			CCDS	Reserved		
								rw	rw	rw	rw	rw			

Бит 3 CCDS (Capture/Compare DMA Selection):
Эти биты управляют выбором источника запроса DMA:

- 0: Запрос DMA не отправляется.
- 1: Запрос DMA отправляется при обновлении события.

Биты 6:4 – MMS (Master Mode Selection):
Эти биты определяют режим управления мастером:

- 000: Режим сброса (Reset) — сигнал сброса генерируется при обновлении.
- 001: Режим включения (Enable) — сигнал включения генерируется при обновлении.
- 010: Режим обновления (Update) — сигнал обновления генерируется при обновлении.
- 011: Режим сравнения (Compare) — сигнал сравнения генерируется при совпадении.

- 100: Режим захвата (Capture) — сигнал захвата генерируется при захвате.
- 101: Режим сравнения канала 1 (Compare Channel 1) — сигнал сравнения канала 1 генерируется при совпадении.
- 110: Режим сравнения канала 2 (Compare Channel 2) — сигнал сравнения канала 2 генерируется при совпадении.
- 111: Режим сравнения канала 3 (Compare Channel 3) — сигнал сравнения канала 3 генерируется при совпадении.

Бит 7 TI1S (TI1 Selection):
Этот бит управляет выбором источника для канала 1:
- 0: Канал 1 подключен к TI1.
- 1: Канал 1 подключен к XOR комбинации TI1, TI2 и TI3 (режим XOR).

TIM_SR
TIM_SR – регистр статуса таймера

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CC4OF	CC3OF	CC2OF	CC1OF	Reserved		TIF	Res	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0			rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

Бит 0 UIF Флаг обновления (Update Interrupt Flag):
- 0: Нет события обновления.
- 1: Произошло событие обновления.

Бит 1 CC1IF Флаг совпадения/захвата канала 1 (Capture/Compare 1 Interrupt Flag):
- 0: Нет события.
- 1: Произошло событие совпадения/захвата канала 1.

Бит 2 CC2IF Флаг совпадения/захвата канала 2 (Capture/Compare 2 Interrupt Flag):
- 0: Нет события.
- 1: Произошло событие совпадения/захвата канала 2.

Бит 3 CC3IF Флаг совпадения/захвата канала 3 (Capture/Compare 3 Interrupt Flag):
- 0: Нет события.
- 1: Произошло событие совпадения/захвата канала 3.

Бит 4 CC4IF Флаг совпадения/захвата канала 4 (Capture/Compare 4 Interrupt Flag):
- 0: Нет события.
- 1: Произошло событие совпадения/захвата канала 4.

Бит 6 TIF Флаг триггера (Trigger Interrupt Flag):
- 0: Нет события триггера.
- 1: Произошло событие триггера.

Бит 9 CC1OF Флаг переполнения захвата канала 1 (Capture/Compare 1 Overcapture Flag):
- 0: Нет переполнения.
- 1: Произошло переполнение захвата канала 1.

Бит 10 CC2OF Флаг переполнения захвата канала 2 (Capture/Compare 2 Overcapture Flag):
- 0: Нет переполнения.

- 1: Произошло переполнение захвата канала 2.

Бит 11 CC3OF Флаг переполнения захвата канала 3 (Capture/Compare 3 Overcapture Flag):

- 0: Нет переполнения.
- 1: Произошло переполнение захвата канала 3.

Бит 12 CC4OF Флаг переполнения захвата канала 4 (Capture/Compare 4 Overcapture Flag):

- 0: Нет переполнения.
- 1: Произошло переполнение захвата канала 4.

TIM_EGR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									TG	Res.	CC4G	CC3G	CC2G	CC1G	UG
									w		w	w	w	w	w

Регистр TIMx_EGR (Event Generation Register) — это регистр генерации событий таймера в микроконтроллерах STM32, включая STM32F103RBT6. Он позволяет программно генерировать события, такие как обновление, захват/сравнение и триггер, без необходимости ожидания их аппаратного возникновения.

Бит 0 UG Генерация события обновления (Update Generation):

- 0: Нет действия.
- 1: Генерация события обновления.

Бит 1 CC1G Генерация события захвата/сравнения канала 1 (Capture/Compare 1 Generation):

- 0: Нет действия.
- 1: Генерация события захвата/сравнения канала 1.

Бит 2 CC2G Генерация события захвата/сравнения канала 2 (Capture/Compare 2 Generation):

- 0: Нет действия.
- 1: Генерация события захвата/сравнения канала 2.

Бит 3 CC3G Генерация события захвата/сравнения канала 3 (Capture/Compare 3 Generation):

- 0: Нет действия.
- 1: Генерация события захвата/сравнения канала 3.

Бит 4 CC4G Генерация события захвата/сравнения канала 4 (Capture/Compare 4 Generation):

- 0: Нет действия.
- 1: Генерация события захвата/сравнения канала 4.

Бит 6 TG Генерация события триггера (Trigger Generation):

- 0: Нет действия.
- 1: Генерация события триггера.

Немного про биты TIM_EGR
UG (Update Generation):

Этот бит позволяет программно сгенерировать событие обновления (Update Event).
При установке этого бита в 1 происходит:

- Перезагрузка регистра предделителя (PSC) и счетчика (CNT).
- Генерация события обновления (если разрешено в регистре TIMx_CR1).
- Бит автоматически сбрасывается аппаратно после выполнения операции.

CC1G, CC2G, CC3G, CC4G (Capture/Compare Generation):
Эти биты позволяют программно сгенерировать события захвата/сравнения для соответствующих каналов.

При установке бита в 1 происходит:

- Для режима сравнения: генерация события совпадения.
- Для режима захвата: генерация события захвата.
- Биты автоматически сбрасываются аппаратно после выполнения операции.

TG (Trigger Generation):
Этот бит позволяет программно сгенерировать событие триггера (Trigger Event).
При установке этого бита в 1 происходит:

- Генерация события триггера.
- Бит автоматически сбрасывается аппаратно после выполнения операции.

TIM_CCMR1 и TIM_CCMR1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Регистр TIMx_CCMR (Capture/Compare Mode Register) — это регистр, который управляет режимами работы каналов захвата/сравнения таймера в микроконтроллерах STM32, включая STM32F103RBT6. Каждый таймер имеет два регистра TIMx_CCMR1 и TIMx_CCMR2, которые отвечают за настройку каналов 1-2 и 3-4 соответственно.

Для режима сравнения (Output Compare Mode)
CCxS (Capture/Compare Selection):
Устанавливает режим работы канала:

- 00: Канал отключен.
- 01: Канал настроен как вход (захват).
- 10: Канал настроен как выход (сравнение).
- 11: Зарезервировано.

OCxFE (Output Compare Fast Enable):
Включает быстрый режим выхода:

- 0: Быстрый режим выключен.
- 1: Быстрый режим включен (изменение состояния выхода происходит сразу при совпадении).

OCxPE (Output Compare Preload Enable):
Включает предзагрузку регистра сравнения:
- 0: Предзагрузка выключена.
- 1: Предзагрузка включена (значение регистра сравнения обновляется только при событии обновления).

OCxCE (Output Compare Clear Enable):
Включает очистку выхода при событии:
- 0: Очистка выключена.
- 1: Очистка включена.

OCxM (Output Compare Mode):
Определяет режим работы выхода:
- 000: Заморозка (выход не изменяется).
- 001: Активный уровень при совпадении.
- 010: Неактивный уровень при совпадении.
- 011: Переключение при совпадении.
- 100: Принудительный неактивный уровень.
- 101: Принудительный активный уровень.
- 110: PWM mode 1.
- 111: PWM mode 2.

Для режима захвата (Input Capture Mode)
CCxS (Capture/Compare Selection):

Устанавливает режим работы канала:
- 00: Канал отключен.
- 01: Канал настроен как вход (захват).
- 10: Канал настроен как выход (сравнение).
- 11: Зарезервировано.

ICxPSC (Input Capture Prescaler):
Устанавливает предделитель для входного сигнала:
- 00: Нет предделителя.
- 01: Предделитель на 2.
- 10: Предделитель на 4.
- 11: Предделитель на 8.

ICxF (Input Capture Filter):
Устанавливает фильтр для входного сигнала:

Значение от 0 до 15 определяет длину фильтра (количество тактов для фильтрации).

TIM_CNT

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Регистр счетчика. Биты 15 - 0 – значение счетчика

TIM_PSC

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Предделитель. Биты 15 - 0 – значение счетчика

Подробное описание регистра
PSC (Prescaler Value):
Этот регистр задает значение предделителя для тактовой частоты таймера.
Значение предделителя определяет, на сколько делится входная частота таймера:
Частота счетчика = Входная частота таймера / (PSC + 1).
Например, если входная частота таймера составляет 72 МГц, а значение PSC = 7199, то частота счетчика будет:

72 МГц / (7199 + 1) = 10 кГц.

Значение предделителя может быть в диапазоне от 0x0000 до 0xFFFF (0 до 65535).

Особенности работы:
Значение предделителя обновляется только при событии обновления (Update Event).
Это означает, что новое значение PSC вступит в силу только после следующего обновления таймера.
Если требуется немедленное обновление, можно программно сгенерировать событие обновления, установив бит UG в регистре TIMx_EGR.

TIM_ARR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Регистр auto-reload (авто-перезагрузки). Биты 15 - 0 – значение перезагрузки
Регистр определяет верхний предел счета таймера и используется для задания периода счета. Когда счетчик таймера достигает значения, записанного в TIMx_ARR, он либо сбрасывается в 0 (в режиме счета вверх), либо начинает считать в обратном направлении (в режиме счета вниз).

Подробное описание регистра
ARR (Auto-Reload Value):
Этот регистр задает верхний предел счета таймера.

Когда счетчик таймера достигает значения ARR, происходит:

- В режиме счета вверх: счетчик сбрасывается в 0.
- В режиме счета вниз: счетчик начинает считать в обратном направлении.

Значение ARR может быть в диапазоне от 0x0000 до 0xFFFF (0 до 65535).

Особенности работы:
Значение ARR может быть предзагружено (буферизировано) с использованием бита ARPE в регистре TIMx_CR1:

- Если ARPE = 0, значение ARR обновляется сразу.
- Если ARPE = 1, значение ARR обновляется только при событии обновления (Update Event).

Для немедленного обновления значения ARR можно программно сгенерировать событие обновления, установив бит UG в регистре TIMx_EGR.

$$T = \frac{1}{f_{CNT}} * (ARR + 1)$$

где:

- f_{CNT}– счетчик таймера;
- ARR – значение регистра перезагрузки.

Допустим тактирование таймера равно 72 МГц, а значение предделителя равно 7199. Тогда f_{CNT} будет равен 10000 Гц (10 кГц, 72000000 / 7199 = 10000).
И тогда, при значении ARR = 9999, период срабатывания таймера будет равен:

$$T = \frac{1}{10000} * (9999 + 1) = 1 \text{ с.}$$

TIM_CCR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro	rw/ro

Регистр захвата/сравнения таймера. Бит 15 - 0 – значение регистра захвата/сравнения таймера.

Регистр TIMx_CCR (Capture/Compare Register) — это регистр захвата/сравнения таймера в микроконтроллерах STM32, включая STM32F103RBT6. Каждый таймер имеет несколько регистров TIMx_CCR1, TIMx_CCR2, TIMx_CCR3 и TIMx_CCR4, которые соответствуют каналам 1, 2, 3 и 4 соответственно. Эти регистры используются для хранения значений сравнения в режиме сравнения (Output Compare) или значений захвата в режиме захвата (Input Capture).

В режиме сравнения (Output Compare Mode):
Регистр TIMx_CCR содержит значение, с которым сравнивается счетчик таймера. Когда значение счетчика совпадает с CCR, генерируется событие совпадения (Compare Event), и в зависимости от настроек может изменяться состояние выхода канала.

В режиме захвата (Input Capture Mode):
Регистр TIMx_CCR содержит значение счетчика, которое было захвачено при возникновении события на входе канала.

Значение CCR может быть в диапазоне от 0x0000 до 0xFFFF (0 до 65535).

Особенности работы:
В режиме сравнения значение CCR может быть предзагружено (буферизировано) с использованием бита OCxPE в регистре TIMx_CCMRx:

- Если OCxPE = 0, значение CCR обновляется сразу.
- Если OCxPE = 1, значение CCR обновляется только при событии обновления (Update Event).

В режиме захвата значение CCR обновляется аппаратно при возникновении события захвата.

На примере генерации ШИМ: если выставить CCR = 4999, то для периода срабатывания таймера равному 1 с, импульс ШИМ будет генерироваться каждые 0.5 сек (коэффициент заполнения 50%).

HAL_TIM_Base_Init

*HAL_StatusTypeDef HAL_TIM_Base_Init(TIM_HandleTypeDef *htim);*

Используется для инициализации базового режима таймера (Timer Base). Эта функция настраивает таймер для работы в режиме счета, без использования дополнительных функций, таких как захват/сравнение или ШИМ.

Параметры

htim: Указатель на структуру TIM_HandleTypeDef, которая содержит конфигурацию таймера.

Возвращаемое значение

HAL_StatusTypeDef: Статус выполнения функции:

- *HAL_OK*: Инициализация выполнена успешно.
- *HAL_ERROR*: Произошла ошибка.

TIM_HandleTypeDef

TIM_HandleTypeDef предназначена для хранения конфигурации и состояния таймера. Эта структура передается в функции HAL для инициализации, управления и обработки таймеров.

```
typedef struct {
   TIM_TypeDef                *Instance;     // Указатель на регистры таймера
   TIM_Base_InitTypeDef        Init;          // Настройки базового режима таймера
   TIM_OC_InitTypeDef          OC_Init;       // Настройки режима сравнения (Output
Compare)
   TIM_IC_InitTypeDef          IC_Init;       // Настройки режима захвата (Input Capture)
   TIM_BDTR_InitTypeDef        BDTR_Init;     // Настройки режима Break and Dead-Time
(для расширенных таймеров)
   HAL_TIM_ActiveChannel       Channel;       // Активный канал таймера
   DMA_HandleTypeDef           *hdma[7];      // Указатели на DMA-каналы для таймера
   HAL_LockTypeDef             Lock;          // Блокировка для защиты от одновременного
доступа
   __IO HAL_TIM_StateTypeDef   State;         // Состояние таймера
   __IO HAL_TIM_ChannelStateTypeDef ChannelState[4]; // Состояние каналов таймера
   __IO HAL_TIM_ChannelStateTypeDef ChannelNState;   // Состояние дополнительного
канала (для расширенных таймеров)
} TIM_HandleTypeDef;
```

Поля структуры

Instance:

- Тип: TIM_TypeDef*
- Описание: Указатель на регистры таймера (например, TIM2, TIM3 и т.д.).

Init:

Тип: TIM_Base_InitTypeDef

Описание: Структура, содержащая настройки базового режима таймера (предделитель, режим счета, период и т.д.).

Поля структуры *TIM_Base_InitTypeDef*:

- Prescaler: Значение предделителя (PSC).
- CounterMode: Режим счета (вверх, вниз или центр-выровненный).

- Period: Значение авто перезагрузки (ARR).
- ClockDivision: Делитель тактовой частоты (CKD).
- RepetitionCounter: Счетчик повторений (используется только в расширенных таймерах).
- AutoReloadPreload: Режим предзагрузки регистра ARR (включен/выключен).

OC_Init:
Тип: TIM_OC_InitTypeDef
Описание: Структура, содержащая настройки режима сравнения (Output Compare). Используется для настройки ШИМ, генерации сигналов и т.д.
Поля структуры TIM_OC_InitTypeDef:
- OCMode: Режим работы канала (например, PWM, Output Compare).
- Pulse: Значение сравнения (CCR).
- OCPolarity: Полярность выходного сигнала.
- OCNPolarity: Полярность дополнительного выходного сигнала (для расширенных таймеров).
- OCFastMode: Быстрый режим выхода.
- OCIdleState: Состояние выхода в режиме ожидания.
- OCNIdleState: Состояние дополнительного выхода в режиме ожидания.

IC_Init:
Тип: TIM_IC_InitTypeDef
Описание: Структура, содержащая настройки режима захвата (Input Capture). Используется для измерения длительности сигналов.
Поля структуры TIM_IC_InitTypeDef:
- ICPolarity: Полярность входного сигнала.
- ICSelection: Выбор источника входного сигнала.
- ICPrescaler: Предделитель входного сигнала.
- ICFilter: Фильтр входного сигнала.

BDTR_Init:
Тип: TIM_BDTR_InitTypeDef
Описание: Структура, содержащая настройки режима Break and Dead-Time. Используется только в расширенных таймерах (например, TIM1, TIM8).
Поля структуры TIM_BDTR_InitTypeDef:
- OSSRState: Состояние выходного сигнала в режиме ожидания.
- OSSIState: Состояние выходного сигнала в режиме ожидания.
- LockLevel: Уровень блокировки.
- DeadTime: Время мертвой зоны.
- BreakState: Состояние функции Break.
- BreakPolarity: Полярность сигнала Break.
- AutomaticOutput: Автоматический выход.

Channel:
Тип: HAL_TIM_ActiveChannel
Описание: Активный канал таймера. Используется для указания, какой канал активен в данный момент.
Возможные значения:

- HAL_TIM_ACTIVE_CHANNEL_1
- HAL_TIM_ACTIVE_CHANNEL_2
- HAL_TIM_ACTIVE_CHANNEL_3
- HAL_TIM_ACTIVE_CHANNEL_4
- HAL_TIM_ACTIVE_CHANNEL_CLEARED

hdma:
Тип: DMA_HandleTypeDef*[7]
Описание: Массив указателей на DMA-каналы, связанные с таймером. Используется для управления передачей данных через DMA.

Lock:
Тип: HAL_LockTypeDef
Описание: Блокировка для защиты от одновременного доступа к таймеру.

State:
Тип: HAL_TIM_StateTypeDef
Описание: Состояние таймера.
Возможные значения:
- HAL_TIM_STATE_RESET
- HAL_TIM_STATE_READY
- HAL_TIM_STATE_BUSY
- HAL_TIM_STATE_TIMEOUT
- HAL_TIM_STATE_ERROR

ChannelState:
Тип: HAL_TIM_ChannelStateTypeDef[4]
Описание: Состояние каналов таймера (для каналов 1-4).

ChannelNState:
Тип: HAL_TIM_ChannelStateTypeDef
Описание: Состояние дополнительного канала (для расширенных таймеров).

HAL_TIM_Base_Start
HAL_TIM_Base_Start используется для запуска базового режима таймера (Timer Base). После вызова этой функции таймер начинает считать в соответствии с настройками, заданными в структуре TIM_HandleTypeDef.
*HAL_StatusTypeDef HAL_TIM_Base_Start(TIM_HandleTypeDef *htim);*
Параметры
htim: Указатель на структуру TIM_HandleTypeDef, которая содержит конфигурацию таймера.

Возвращаемое значение: HAL_StatusTypeDef: Статус выполнения функции:
- HAL_OK: Таймер успешно запущен.
- HAL_ERROR: Произошла ошибка.

HAL_TIM_Base_Stop
*HAL_StatusTypeDef HAL_TIM_Base_Stop(TIM_HandleTypeDef *htim);*

HAL_TIM_Base_Stop используется для остановки базового режима таймера (Timer Base). После вызова этой функции таймер прекращает счет, и его состояние переходит в HAL_TIM_STATE_READY.

Параметры
htim: Указатель на структуру TIM_HandleTypeDef, которая содержит конфигурацию таймера.
Возвращаемое значение
HAL_StatusTypeDef: Статус выполнения функции:

- HAL_OK: Таймер успешно остановлен.
- HAL_ERROR: Произошла ошибка.

__HAL_TIM_SET_COUNTER
__HAL_TIM_SET_COUNTER(__HANDLE__, __COUNTER__)
Данный макрос устанавливает значение счетчика таймера
__HANDLE__ – указатель на структуру таймера (htim);
__COUNTER__ – значение счетчика таймера.

Реализация макроса:
((__HANDLE__)->Instance->CNT = (__COUNTER__))
Пример вызова:
__HAL_TIM_SET_COUNTER(&htim2, 1000);

__HAL_TIM_GET_COUNTER
__HAL_TIM_GET_COUNTER(__HANDLE__)
Данный макрос считывает текущее значение счетчика таймера
__HANDLE__ – указатель на структуру таймера (htim#).

Пример вызова:
__HAL_TIM_GET_COUNTER(&htim2);

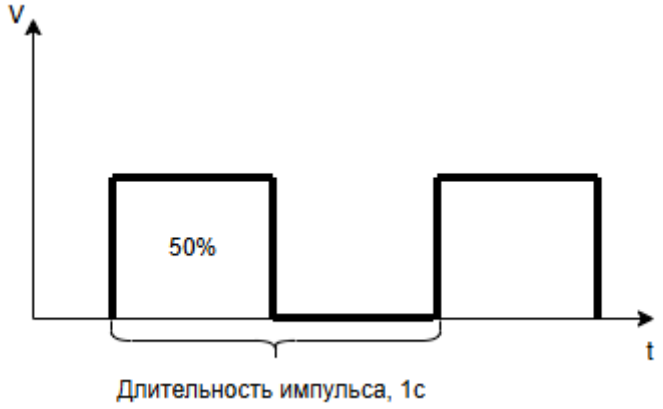
Задание:

1. Реализуйте моргание светодиодом (можно использовать как светодиод на плате, так и внешний) по задержке таймера.
   Должны быть реализованы следующие варианты задержки таймера:
   - 100 милисекунд;
   - 500 милисекунд;
   - 1 секунда.

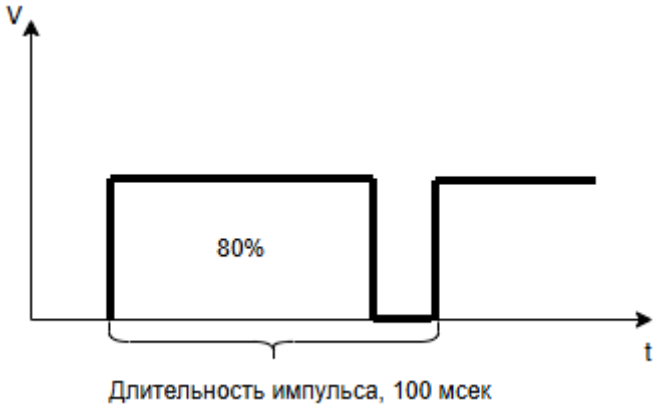
Внимание: нельзя использовать функцию HAL_Delay для организации задержки. Используйте ваш настроенный таймер (TIM2, TIM3 и т.д.)

2. Реализовать мигание светодиодом по прерыванию таймера. Должны быть реализованы следующие варианты прерываний:
   - прерывание по переполнению счетчика таймера;
   - прерывание по спадающему фронту (с 1 до 0) импульса ШИМ.

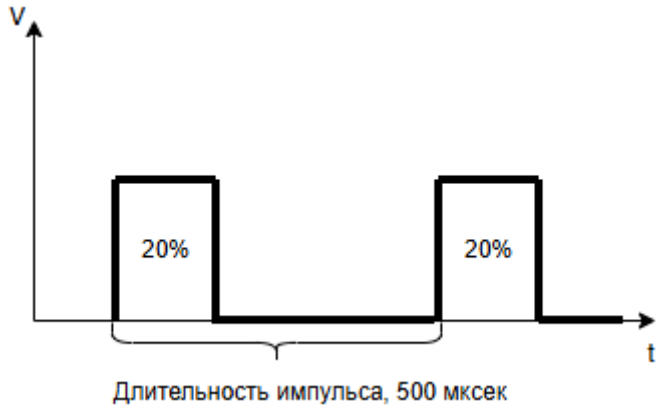
3. Реализуйте генерацию ШИМ, который выдает следующие варианты импульсов:
   а) длительность импульса – 1 сек. Скважность – 50%.



   б) длительность импульса – 100 мсек. Скважность – 80%.



   в) длительность импульса – 500 мксек. Скважность – 20%.



Внимание: если в вашей версии STM32CubeIDE нельзя задавать настройку параметра Pulse через ioc-файл, добавьте в код перед запуском таймера (HAL_TIM_[type]_Start()):

TIM2->CCR2 = 499

Пояснение:

- 499 – значение заполнения (помещается в регистр сравнения CCR2);
- если вы выбрали другой таймер для работы (не TIM2), то вместо TIM2 поставьте номер своего таймера;
- если вы выбрали другой канал для работы таймера (не Channel 2), то для CCR вместо 2 поставьте номер своего канала.

4. Необязательно: с генератора ШИМ-сигнала подайте на вход таймера прямоугольный импульс, и реализуйте захват переднего и/или заднего фронтов в режиме Input Capture
5. Необязательно: рассчитайте период приходящего на вход таймера прямоугольного импульса.
6. Представьте выполнение шагов 1-3 (опционально 4-5) в отчёте. Также прикрепите в задании файл main.c с вашим кодом выполнения заданий 1-3 (опционально 4-5).